Lab 2: Exchange messages with MQTT

1. MQTT Communication with Python

MQTT is an extremely lightweight messaging protocol for the Internet of Things (see here: https://mqtt.org/). It has been widely supported by the Amazon cloud, Azure cloud, etc. Fig. 1 shows the network architecture of the MQTT protocol. The network is composed of publishers, MQTT brokers, and subscribers:

- Publishers: data providers (can be IoT devices, like Raspberry Pi)
- Subscribers: data consumers (such as a monitoring application)
- MQTT broker: the bridge between publisher and subscriber

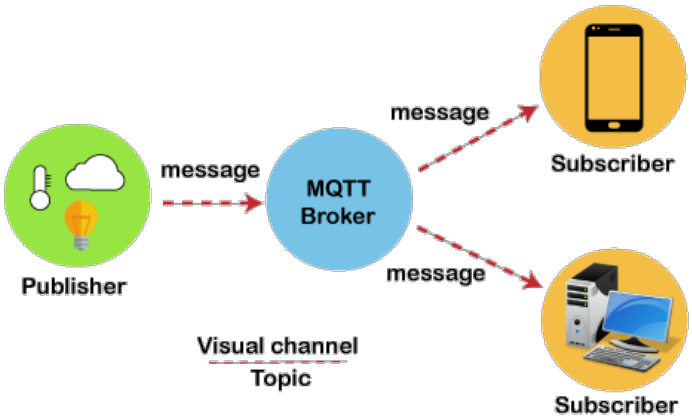


Fig. 1. Architecture of MQTT

Here, we will not go through the details of the MQTT protocol. However, we can still do MQTT experiments for message exchange since there are well-developed MQTT libraries in different languages, etc. One well-known tool is Eclipse mosquitto, which is an open-source MQTT broker. A free and ready-to-use MQTT broker is deployed here: https://test.mosquitto.org/.

Let us set up an experiment to see how it works. Install mosquitto using the following command in the terminal of Linux.

sudo apt-get install mosquitto
sudo apt-get install mosquitto-clients

Let's open one terminal and use the following command to subscribe to the topic (tecnun/IAI) on test.mosquitto.org in the terminal:

mosquitto_sub -h test.mosquitto.org -t tecnun/IAI

Use the following command to publish a message to the same topic (tecnun/IAI):

mosquitto_pub -h test.mosquitto.org -t tecnun/IAI -m "Hello from Tecnun!"

Collaborate with the other groups and see if you can exchange messages with them.

Up to now, we have practiced how to exchange MQTT messages manually. However, we have learned programming, e.g. Python. So how to send MQTT messages with Python code? Fortunately, we have an open-source library called paho-mqtt (https://pypi.org/project/paho-mqtt/) for MQTT communication. Utilizing this library, we can write Python code to send MQTT messages. To start, let us first install the Python library with the command:

pip3 install paho-mqtt

(Note: If you use the command 'pip install paho-mqtt', the library is installed for Python version 2, not for Python version 3. So, it depends on which Python version you are using for programming.)

Then, you can refer to the code below, which automatically connects to the remote MQTT server of Eclipse, gets messages from user, and publish the input to the given MQTT topic.

```
import paho.mqtt.client as mqttClient
import time

def on_connect(client, userdata, flags, rc):
    if rc==0:
        print("Connected to broker")
        global Connected # global variable defined later
        Connected = True
    else:
        print("Connected failed")

Connected = False #global variable for the state of the connection

broker_address = "test.mosquitto.org" #set your broker address
port = 1883  # listening port of the mqtt broker
#user = "tecnun"  # uncomment this to set your username if needed
#password = "masterAI"  # uncomment this set the password if needed
clientiD = "IAI" # set the client ID
topic = "tecnun/IAI" # set you mqtt topic

client = mqttClient.Client(clientID) # creat mqtt object
#client.username_pw_set(user, password) # uncomment this if you have
username and password
client.on_connect = on_connect
client.connect(broker_address, port=port)

client.loop_start() #start the loop


while Connected != True:
    time.sleep(1)
    print("waiting to connect")
```

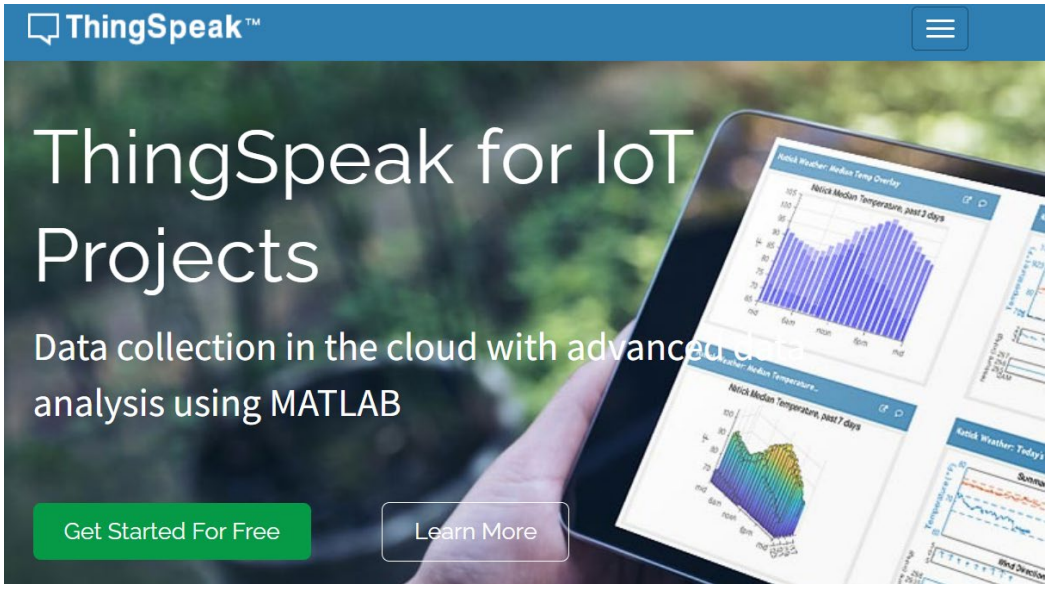
```
try:
    while True:
        print('Enter a numeric value:')
        value = input()
        client.publish(topic, value)  #publish to the topic "tecnun/IAI"

except KeyboardInterrupt:
    client.disconnect()
    client.loop_stop()
```

Try the code and see if you can run it successfully to publish a message and receive the message with mosquitto as we did before. The message may also be received by the other groups if the same MQTT topic is used.

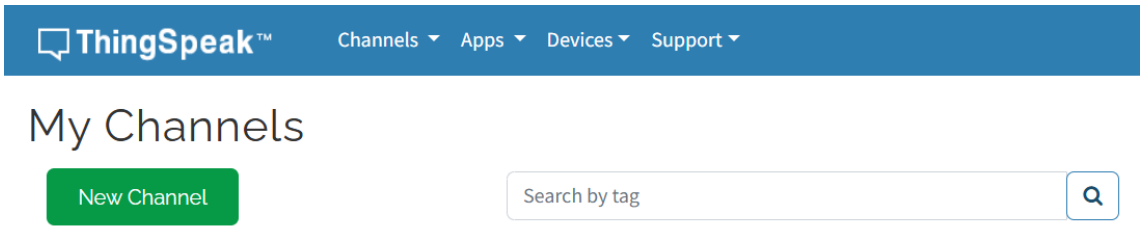
2. Sending data to the cloud with MQTT

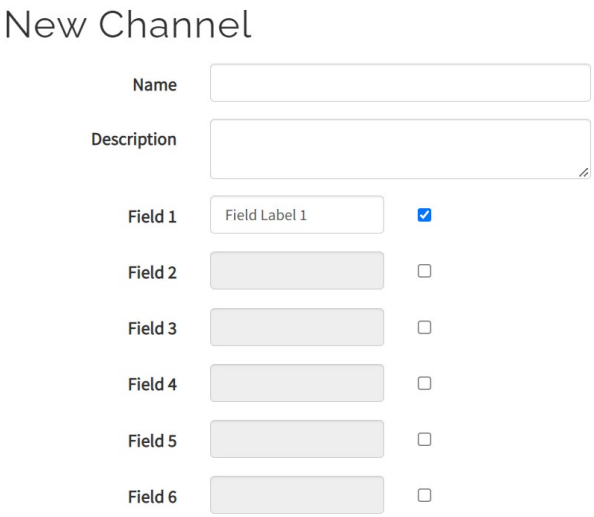
Due to the easy-to-use feature, MQTT has been integrated into many cloud systems, such as Azure and Amazon, to receive data from IoT devices. Here, we will do an experiment to send manual data (later you can change this to real sensor data) to the ThingSpeak cloud (https://thingspeak.com/). ThingSpeak is specially designed for IoT education and is free to use for students.

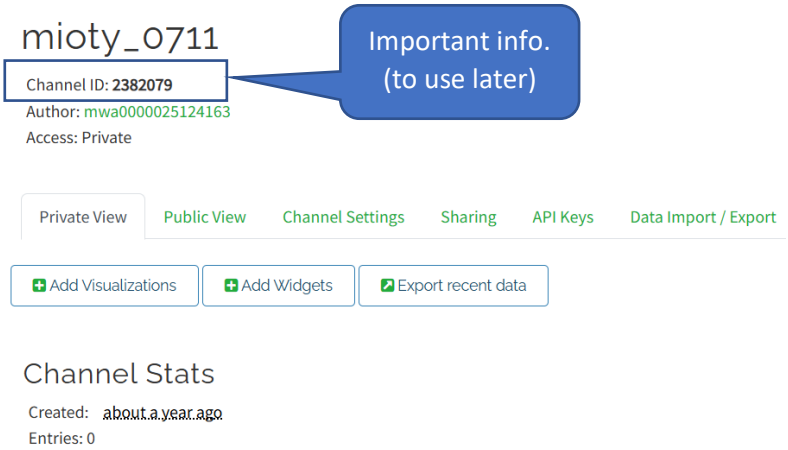


To send IoT data to ThingSpeak, you will need to do the following:

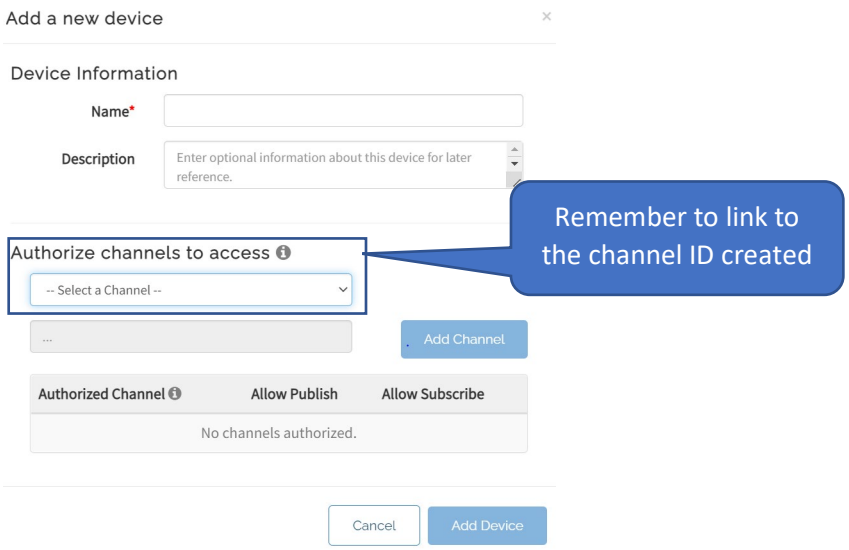
- Create an account
- Create a 'Channel' to host data (Channels → My Channels→New Channel. It is OK to use the default setting.)



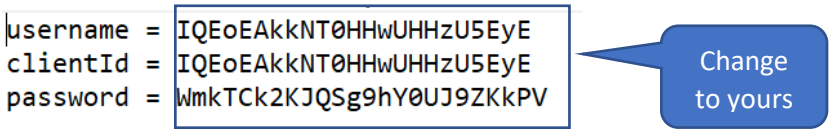




- The next step is to create a Device based on MQTT. Click the 'Devices' on top, Set the device a name you like, and link it to the 'Channel' that you created recently.



After the device is added to ThingSpeak, it will pop up a dialog for you to save the credentials, including a **username**, a **clientId**, and a **password** (you can save it as a plain text file). The information is important to connect and send data to ThingSpeak.



More information about the MQTT at ThingSpeak is provided here (https://www.mathworks.com/help/thingspeak/mqtt-basics.html). Going through this page, we can find some important information:

- The hosting address of Thingspeak MQTT broker is **mqtt3.thingspeak.com** on the TCP port **1883**.
- The **topic** is designed in the format: **channels/<channelID>/publish** (replace `<channelID>` with your channel ID)
- The message format is **field1=<value1>&field2=<value2>** (replace <valueX> with a number)

Now referring to the Python code developed for MQTT message exchange before, modify it to publish data to ThingSpeak.

Practice task:

MQTT is a very useful protocol for IoT. In this lab, we have been using the online MQTT brokers of mosquitto and ThingSpeak. However, online MQTT brokers are not so efficient when it is only required to share data with neighboring devices in a local network. In this case, we may need a local MQTT broker.

Google how to deploy a local mosquito MQTT broker on Raspberry PI. We will use the local MQTT broker in future labs.

Here are the requirements:

- Each group sets up a local MQTT broker on your Raspberry Pi device.
- Collaborate with the other groups to test your MQTT broker, e. g. asking them to exchange messages through your local MQTT broker.
- Document how the MQTT broker was set up step by step.

Note: After the local deployment, you may encounter a ‘connection refused’ issue. The reason is because:

> “*In Mosquitto 2.0 and up, you must choose your authentication options explicitly before clients can connect. In earlier versions, the default is to allow clients to connect without authentication.*“

Refer to the file named ‘Mosquitto remote connection configuration’ to solve it.